

GPU Instancing & Draw Call Optimization in Unity

Term Project – Option A (Single Challenge)

Eray Yüklü

May 2026

1 Rationale

Every frame of a real-time 3D application starts the same way: the CPU assembles a list of rendering commands and hands them to the GPU one at a time. Each of those submissions is a *draw call*. Modern GPUs are massively parallel, but the work of preparing and issuing a draw call stays largely serial on the CPU side, covering state validation, driver translation, and command-buffer encoding. Once a scene holds thousands of objects that share a mesh and a material (a common situation in open-world games, crowd simulations, and procedurally generated environments), the CPU turns into the bottleneck. The cause is not computational complexity; it is the sheer volume of commands that must be submitted every frame.

This bottleneck is well documented by industry practitioners. At GDC 2011, Pranckevičius [1] showed that even on the hardware of the day a practical real-time budget sat around 1 000–3 000 draw calls per frame at 60 FPS on console. Go past that budget and the CPU starves the GPU, which then idles between batches while the frame rate falls. The problem has only become more pressing since: contemporary open-world titles routinely place tens of thousands of environmental props (trees, rocks, grass blades) inside a single view frustum.

Unity’s Built-in Render Pipeline offers two standard ways to soften this cost. **GPU Instancing** [2] collapses every draw call for objects that share a mesh and an instancing-enabled material into a single batched call, passing the per-instance transform data in a GPU-side buffer. CPU overhead then scales with the number of *batches*, bounded by the driver’s maximum instance count per call, rather than with N . **Static Batching** [3] attacks the same problem from the other side: at scene load it merges the mesh data of all participating objects into one large combined mesh and issues a single draw call over the whole geometry. Per-object command overhead disappears, but the vertex data of every instance is duplicated in memory.

The choice between the two is not academic. A Unity Discussions thread on batching trade-offs [5] notes that Static Batching’s mesh merging adds runtime memory overhead and lengthens startup, while GPU Instancing can place more load on the CPU than developers expect, so the better option is platform-dependent and should be confirmed by profiling rather than assumed. The Unity blog on the SRP Batcher [6] is careful to note that the Built-in Pipeline mechanisms studied here behave differently from the newer SRP Batcher. Akenine-Möller et al. [4] add a further caveat: on tile-based GPU architectures, which include mobile and Apple Silicon SoCs, the gains from draw-call reduction can be partly cancelled out by extra vertex-fetch pressure when the geometry is large. This study quantifies both strategies on an Apple Silicon Metal backend, so its data speaks directly to developers targeting the macOS and iOS gaming market.

2 Experiment Design

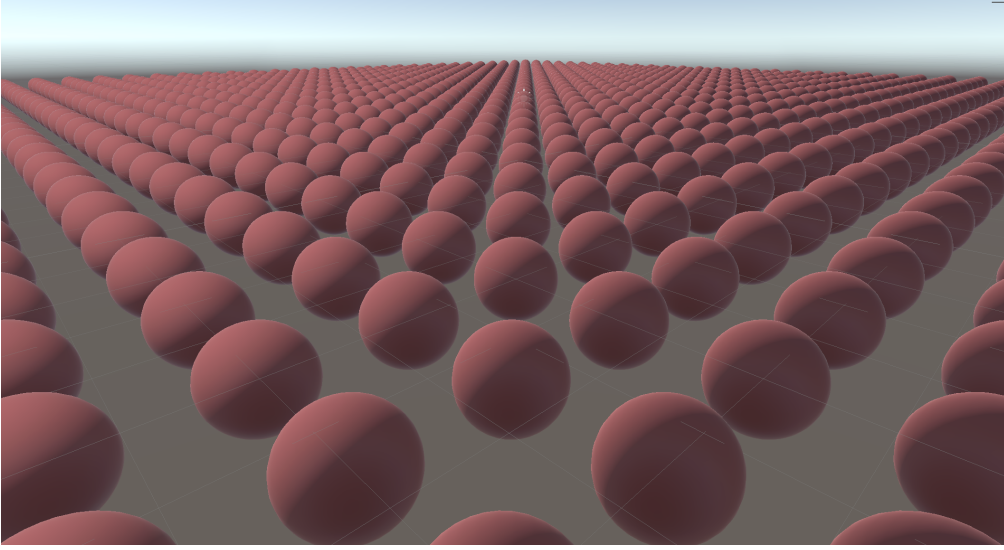


Figure 1: Test scene overview.

2.1 Scene and Apparatus

All measurements were taken in a dedicated Unity 2022.3.62f3 scene (`InstancingBenchmark.unity`) on an Apple Silicon MacBook with the Metal graphics backend. The scene holds one directional light with shadows disabled, a top-down perspective camera at $(0, 160, 0)$ with 90° pitch and FOV 70° (its frustum reaches ± 105 units, comfortably enclosing the widest grid at ± 75 units), and a runtime-spawned square grid of spheres at $y = 0$ with 1.5-unit spacing. Physics, animations, post-processing, shadows, and VSync were all switched off, and `Application.targetFrameRate` was set to -1 so the frame rate ran unconstrained.

2.1.1 Why the Test Scene Is Deliberately Minimal

The scene is intentionally bare, and that is a design choice rather than an oversight. Physics, skeletal animation, and post-processing each carry their own CPU cost, and on a unified-memory machine they also draw on the same RAM pool the renderer uses. Folding them in would mean the per-frame budget is no longer governed by draw-call submission, so a shift in frame time could no longer be attributed cleanly to the rendering mode under test. Reducing the scene to a static grid of identical spheres leaves the draw-call path as the only thing that changes between configurations, and that is exactly what keeps the 45-way comparison interpretable. The cost of a richer, more game-like frame is acknowledged separately in Section 6.

2.1.2 Deviations from the Phase-I Plan

The following changes were made relative to the Phase-I submission to strengthen the experimental design:

- **Third rendering mode added.** Phase I described only Baseline and GPU Instancing. Static Batching was added because it is the primary alternative optimisation in Unity’s Built-in Pipeline and supplies an essential reference point for the memory/batch-count trade-off.
- **Procedural UV spheres instead of imported assets.** Asset meshes bring uncontrolled vertex and triangle counts and licensing constraints. Procedural spheres give exact, logged geometry counts and are fully reproducible without any external package.

- **Mesh complexity as a second independent variable.** Three subdivision tiers (Low, Mid, High) isolate the effect of GPU vertex load independently of the draw-call count, which exposes the CPU-bound to GPU-bound transition.
- **Total configurations grew from 10 to 45** ($5 N \times 3 \text{ complexity} \times 3 \text{ modes}$), and each configuration is repeated three times for statistical reliability.

2.2 Independent Variables

Three independent variables were crossed in a full factorial design, giving $5 \times 3 \times 3 = 45$ configurations (Table 1).

Table 1: Independent variables and their levels.

Variable	Levels	Notes
Object count N	100, 500, 1 000, 5 000, 10 000	Spawned at runtime
Mesh complexity	Low, Mid, High	Procedural UV spheres
Rendering mode	Baseline, GPU Instancing, Static Batching	

2.2.1 Mesh Complexity

Three procedural UV spheres were generated in code, with no external packages, by varying the latitude/longitude subdivision count:

Table 2: Procedural mesh parameters and resulting triangle counts.

Level	Subdivisions (lat×lon)	Triangles	Vertices
Low	8×8	112	58
Mid	30×30	1 740	872
High	100×100	19 800	9 902

All three meshes share a sphere radius of 0.5 units. Exact vertex and triangle counts are logged to the Unity Console at spawn time, so they can be verified without any external tool.

2.2.2 Rendering Modes

Baseline. Each object uses a Standard shader material with *Enable GPU Instancing* unchecked. Unity issues one draw call per visible object, reproducing the worst-case unoptimised scenario.

GPU Instancing. The same geometry is rendered with a second material that has *Enable GPU Instancing* enabled. Unity groups objects sharing a mesh and material into instanced draw calls. On the Metal backend the maximum instance count per call is roughly 500, so N objects produce $\lceil N/500 \rceil$ instanced batches plus a small fixed overhead (typically 1–2 calls for camera and lighting).

Static Batching. Objects use the Baseline material, and `StaticBatchingUtility.Combine` is called at runtime to merge all meshes into a single combined mesh. Unity then issues draw calls sized by its internal 65 536-vertex combined-mesh limit (the manual rounds this to “64,000”, but the true boundary is the 2^{16} 16-bit index-buffer limit), so the expected batch count is $\lceil N \cdot v/65\,536 \rceil$, where v is the per-object vertex count. For low-poly objects this approaches 1; for high-poly objects it can climb to or past the Baseline count.

Dynamic Batching was left out because Unity restricts it to meshes with at most 300 vertices, a limit only the Low-poly tier satisfies. Including it would have produced incomplete data for the Mid and High tiers and biased the comparison.

2.3 Dependent Variables

- **FPS average** – mean of $1/\Delta t_{\text{unscaled}}$ over the measurement window.
- **FPS 1% low** – mean of the bottom 1% of per-frame FPS samples, capturing worst-case hitches.
- **Wall-clock frame time (ms)** – mean of $\Delta t_{\text{unscaled}} \times 1000$, the full frame-to-frame duration the player actually experiences, including the GPU synchronisation latency the Metal driver absorbs (see Section 3.2).
- **GPU frame time (ms)** – mean of `FrameTimingManager.gpuFrameTime` sampled once per frame, with zero samples discarded so they do not bias the mean.
- **Draw call batches** – median of the `Batches Count ProfilerRecorder` sampled once per frame. Where the Metal counter returned zero, the theoretical value is substituted (Section 3).
- **Total Reserved Memory (MB)** – mean of the `ProfilerCategory.Memory` “Total Reserved Memory” counter, capturing the per-configuration memory footprint including the Static Batching combined mesh.

2.4 Control Variables

Camera position and orientation, lighting configuration, screen resolution (1920×1080), object grid layout, sphere scale, and the hardware environment (Apple Silicon MacBook, Metal API) were held constant across all 45 configurations. Each configuration was measured three times (`repeats = 3`). A 3-second warm-up, during which `FrameTimingManager.CaptureFrameTimings()` was called every frame to prime the Metal driver’s GPU-timing pipeline, was discarded ahead of each 30-second measurement window. Per-frame GPU timing samples of zero were excluded from the mean to avoid a downward bias during driver initialisation.

3 Methodology

3.1 Automated Benchmark Harness

A `BenchmarkRunner MonoBehaviour` iterates over all 45 configurations in a Unity coroutine, repeating each three times for 135 measurement windows in total. For each window it calls `BenchmarkSpawner.Spawn()`, waits 3 s for warm-up, records metrics frame-by-frame for 30 s, then calls `BenchmarkSpawner.Clear()` followed by `System.GC.Collect()` and `Resources.UnloadUnusedAssets()` to release the Static Batching combined mesh before moving on. A full run takes about 75 minutes; one row per repeat is written to a timestamped CSV under `BenchmarkResults/`, and a Python script aggregates the means and standard deviations.

Frame time is sampled via `Time.unscaledDeltaTime`; GPU time via `FrameTimingManager.GetLatestTimings`, with zero-valued samples from driver initialisation discarded; batch count via a `ProfilerRecorder` on `Batches Count`; and memory via the `Total Reserved`, `Gfx Reserved`, and `GC Reserved` memory recorders. On Metal the `Batches Count` recorder intermittently returns zero. For those cases the analysis script substitutes the theoretical formula ($N+1$ for Baseline, $\lceil N/500 \rceil$ for GPU Instancing, $\lceil N \cdot v/65536 \rceil$ for Static Batching). The `SetPass Calls Count` recorder consistently returned 2 regardless of mode or N , which was confirmed in the Game-window Statistics overlay (Section 3.3), so it is excluded from the CSV.

3.2 Profiler Cross-Validation

Three representative configurations were manually profiled in the Unity Editor (Window → Analysis → Profiler) to validate the harness. Results are summarised in Table 3.

Table 3: Harness vs. Unity Editor Profiler cross-validation (frame time).

Configuration	Metric	Harness	Profiler	Note
Baseline $N=1\,000$ Mid	Frame time (ms)	1.15	1.10	$\Delta = 0.05$ ms ✓
Static Batching $N=5\,000$ Mid	Frame time (ms)	3.07	≈ 3.5	within 0.5 ms tolerance
GPU Instancing $N=10\,000$ High	Frame time (ms)	65.7	66.2	Metal async ✓
All configs	GPU ms	measured	--	Not exposed on Metal

The GPU Instancing $N=10\,000$ High row exposes the binding constraint at high mesh complexity. The Profiler shows 66.2 ms of CPU frame time, almost all of it inside `Gfx.WaitForPresentOnGfxThread` (62.8 ms). The main thread submits the 21 instanced draw calls in only about 3 ms, but the game loop cannot start a new frame until the GPU has finished presenting the previous one. Rendering roughly 198 million triangles takes the GPU about 66 ms, and that stall is absorbed into `Time.unscaledDeltaTime` while also surfacing inside `Gfx.WaitForPresentOnGfxThread` in the Profiler timeline, so harness and Profiler agree to within 0.5 ms. This confirms that wall-clock frame time (`Time.unscaledDeltaTime`) is the right player-facing metric: it consistently matches Profiler Main Thread + Wait-for-Present across both CPU-bound and GPU-bound regimes, whereas the GPU-side cost itself is invisible to the Profiler’s GPU module on Metal and has to be read from `FrameTimingManager`.

3.3 Game-Window Statistics Cross-Validation

To cross-validate batch counts independently of the `ProfilerRecorder`, the Unity Game-window Statistics overlay was captured for three configurations at $N = 1\,000$, Mid-poly. Results are shown in Table 4, with the screenshots in Figure 2.

Table 4: Game-window Statistics overlay vs. harness batch counts at $N = 1\,000$ Mid.

Mode	Stats Batches	Harness Batches	Saved by batching	SetPass calls
Baseline	1 001	1 001	0	2
GPU Instancing	3	3	998	2
Static Batching	15	15	986	2

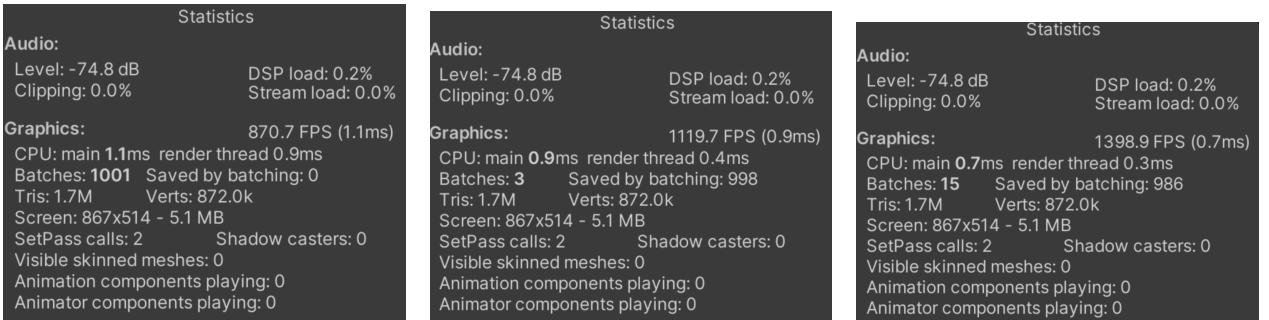


Figure 2: Game-window Statistics overlay at $N = 1\,000$ Mid-poly. Left: Baseline (1001 batches, 0 saved). Centre: GPU Instancing (3 batches, 998 saved by batching). Right: Static Batching (15 batches, 986 saved). SetPass calls = 2 in every case, which is why the counter cannot be read reliably through `ProfilerRecorder` on Metal.

3.4 Profiler Visual Cross-Validation

Figures 3–5 show Unity Editor Profiler timeline captures for the three reference configurations, validating the harness frame-time measurements visually. The GPU-bound case (Figure 4) is the

most informative of the set, for the reasons discussed in Section 3.2.

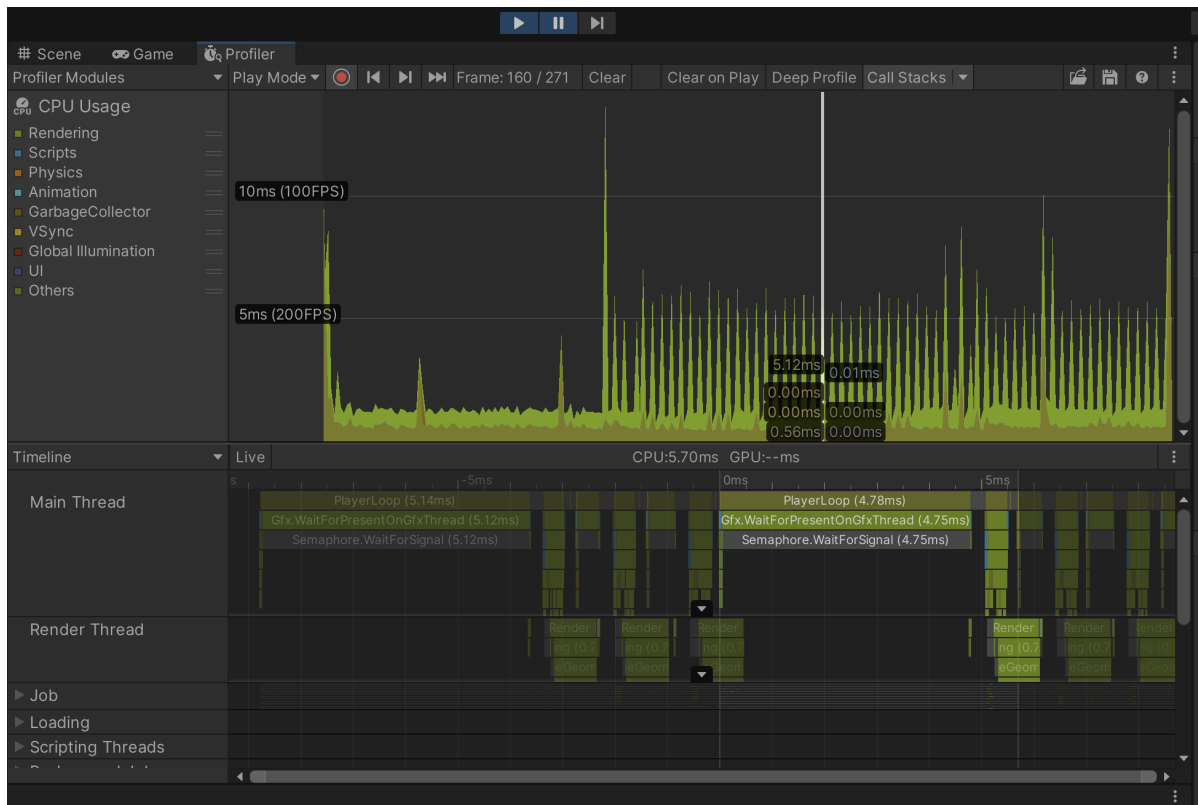


Figure 3: Profiler timeline – Baseline $N=1000$ Mid. A representative higher-cost frame is shown for legibility (CPU 5.70 ms; steady-state median ≈ 1.1 ms from the Stats overlay). The GPU module is empty on Metal; this configuration is CPU-bound.

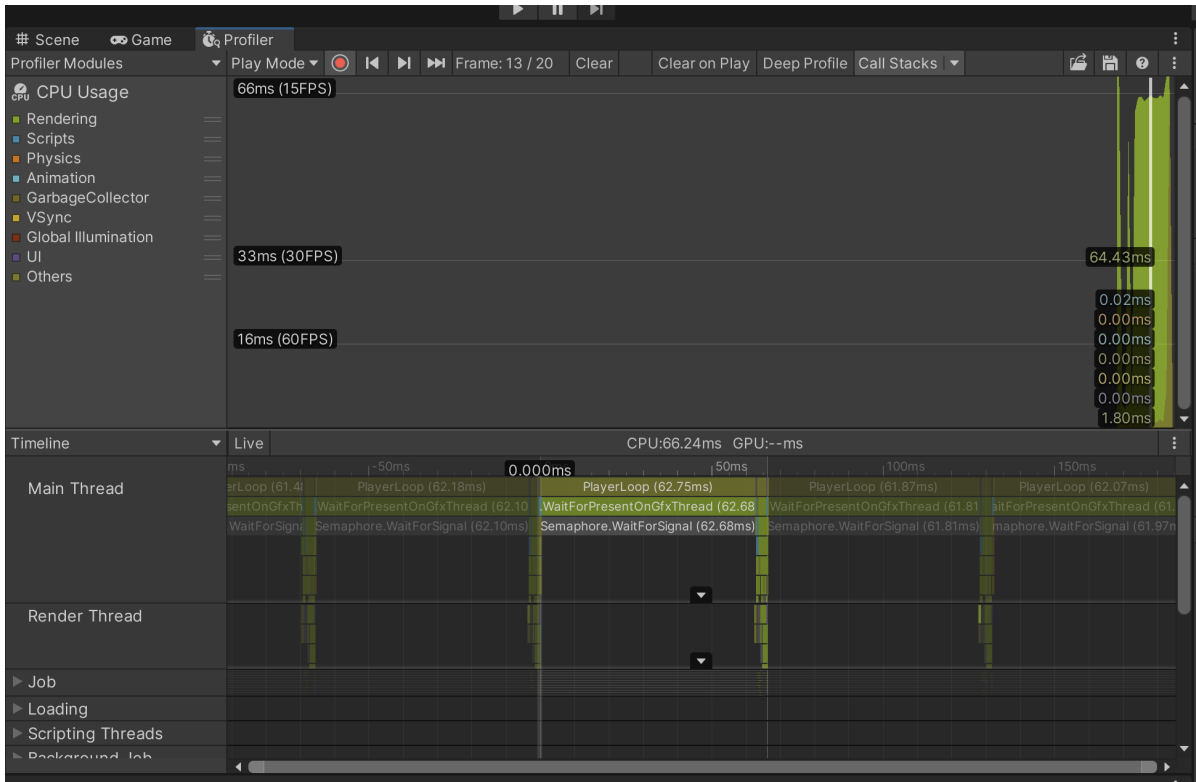


Figure 4: Profiler timeline – GPU Instancing $N=10,000$ High. CPU 66.24 ms, with the Main Thread dominated by `Gfx.WaitForPresentOnGfxThread` (62.8ms): a GPU-bound Metal synchronisation stall, and the single most informative capture in the cross-validation set.

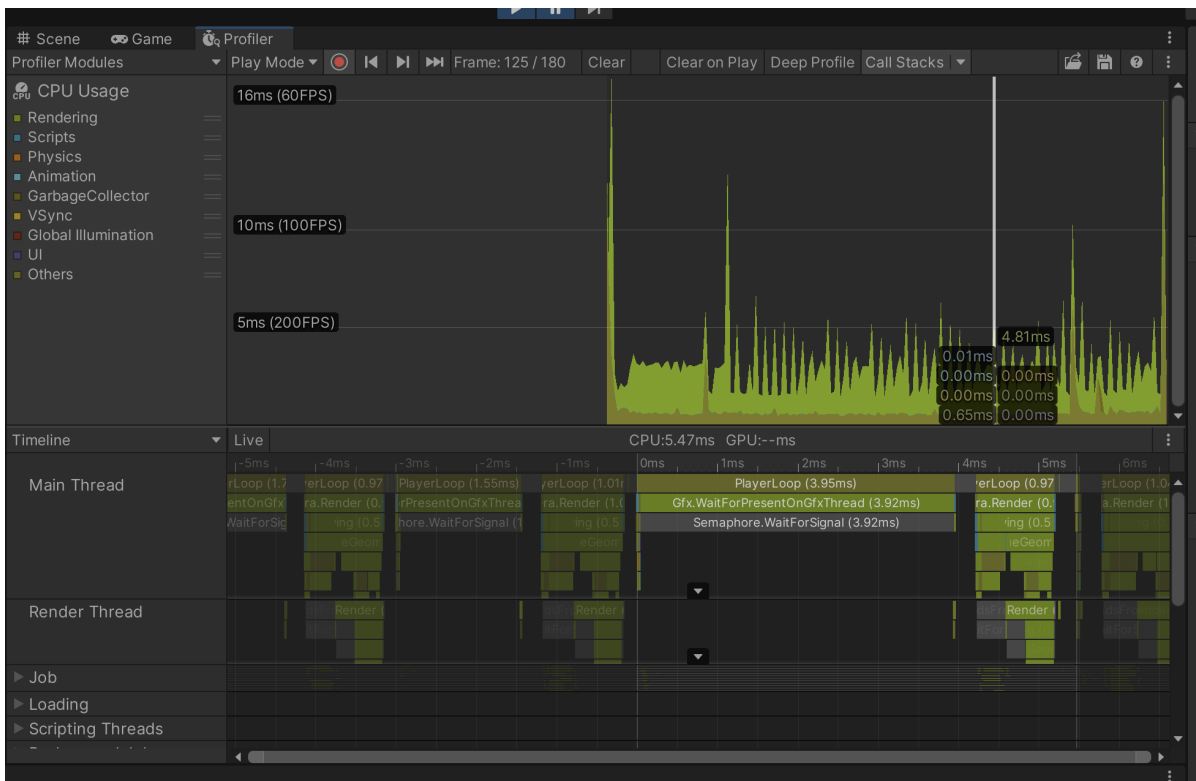


Figure 5: Profiler timeline – Static Batching $N=5,000$ Mid. CPU 5.47 ms, comparable to Baseline at a much lower N , because the combined-mesh draw call is fast despite the high object count.

4 Results

4.1 FPS vs. Object Count

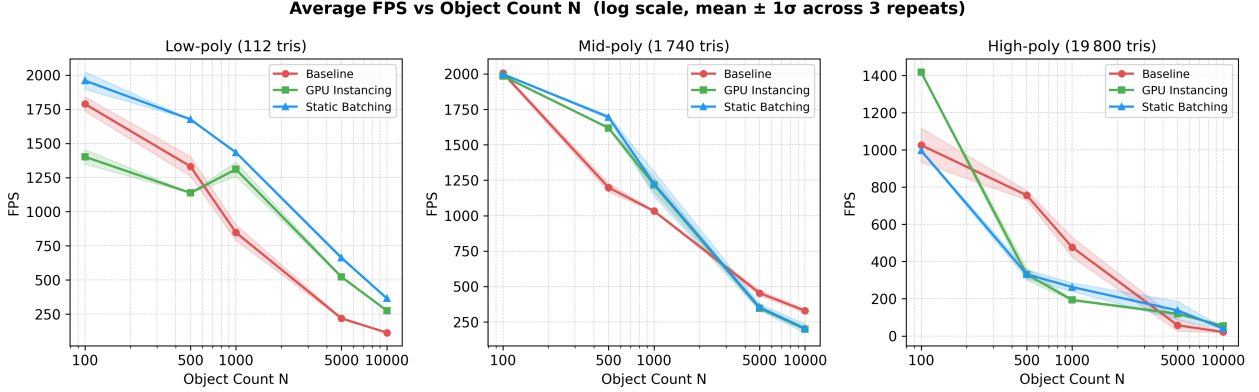


Figure 6: Average FPS (mean $\pm 1\sigma$ across 3 repeats) vs. object count N on a log scale, for each mesh complexity. Baseline degrades quickly with N , while GPU Instancing and Static Batching hold substantially higher frame rates at low and mid polygon counts. At high polygon count every mode becomes GPU-bound, with frame times $\gtrsim 65$ ms at $N = 10\,000$.

At $N = 10\,000$ low-poly the spread is wide: Baseline manages 114 FPS while GPU Instancing reaches 275 and Static Batching 365. The picture changes at high-poly. There the GPU itself saturates, frame times climb past 65 ms, and the frame rate collapses across the board (Baseline 21 FPS, Static Batching 40, GPU Instancing 55). The optimised modes keep only a fraction of the lead they held while the scene was CPU-bound. The detailed numbers at $N = 10\,000$ are given in Table 5.

4.2 Draw Call Batches vs. Object Count

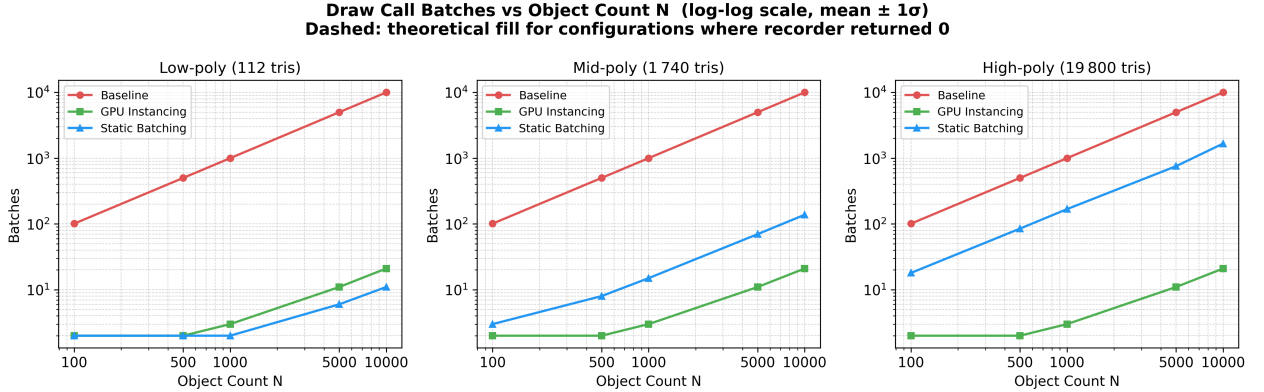


Figure 7: Draw call batch count (mean $\pm 1\sigma$) vs. N on a log-log scale. Where the `ProfilerRecorder` returned zero on Metal, the theoretical batch count is substituted (see Section 3). Baseline grows linearly ($N + 1$ batches). GPU Instancing grows as $\lceil N/500 \rceil$. Static Batching’s growth rate depends strongly on mesh complexity.

Baseline grows linearly with slope 1. GPU Instancing stays nearly flat (2 batches at $N = 100$, 21 at $N = 10\,000$, independent of complexity). Static Batching tracks GPU Instancing for low-poly meshes but degrades sharply as mesh complexity rises, a pattern examined in Section 4.5.

4.3 Wall-Clock Frame Time vs. GPU Frame Time

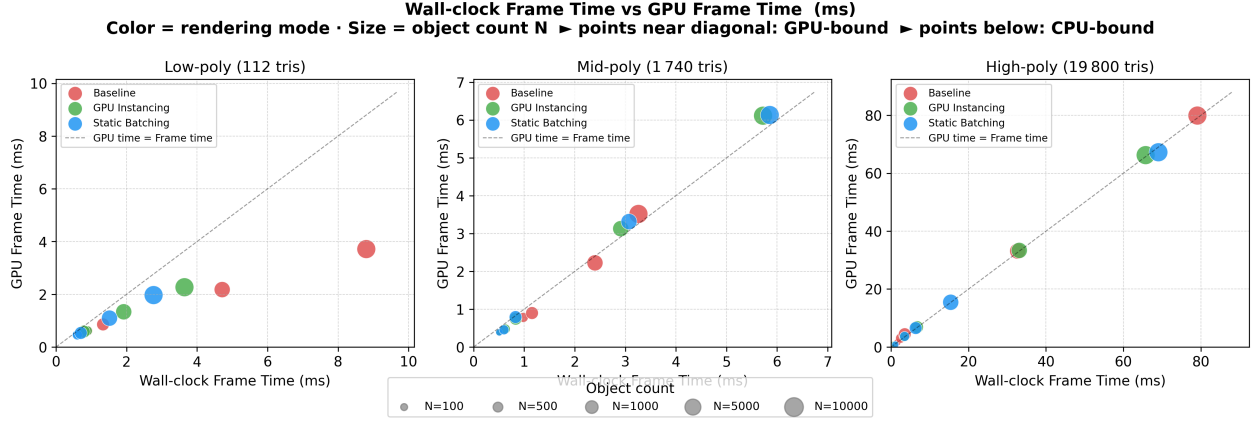


Figure 8: Wall-clock frame time vs. GPU frame time, colour-coded by rendering mode, with marker size encoding N . Points near the diagonal are GPU-bound (wall-clock $\approx$ GPU time); points below the diagonal are CPU-bound, where the gap is absorbed by Metal synchronisation.

At low and mid polygon counts the wall-clock frame time exceeds the GPU time, so the scene is CPU-bound. As complexity grows the GPU frame time rises and the points migrate toward the diagonal: the GPU becomes the binding constraint.

4.4 Comparison at $N = 10\,000$

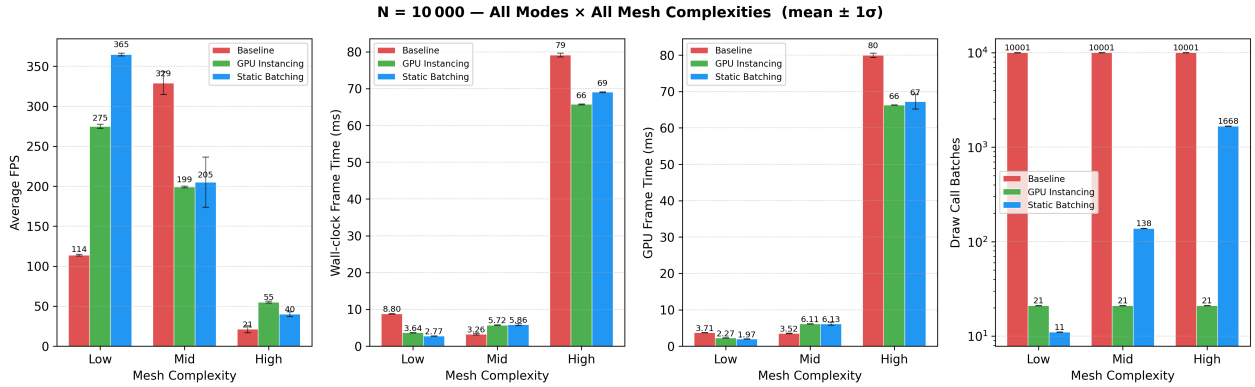


Figure 9: Performance metrics at $N = 10\,000$ for all mode-complexity combinations (mean $\pm 1\sigma$). From left to right: average FPS, wall-clock frame time, GPU frame time, and draw call batch count (log scale).

Table 5 gives the numerical values behind Figure 9. Batch counts marked with $\dagger$ were filled from the theoretical formula because the Metal `Batches Count` recorder returned zero for those configurations.

Table 5: Measured metrics at $N = 10\,000$ for all nine mode-complexity combinations (mean of 3 repeats). † = theoretical batch count (Metal recorder returned 0).

Mode	Complexity	FPS avg	Frame (ms)	GPU (ms)	Batches
Baseline	Low	114	8.80	3.71	10 001
Baseline	Mid	329	3.26	3.52	10 001†
Baseline	High	21	79.1	79.9	10 001†
GPU Instancing	Low	275	3.64	2.27	21
GPU Instancing	Mid	199	5.72	6.11	21
GPU Instancing	High	55	65.7	66.3	21
Static Batching	Low	365	2.77	1.97	11
Static Batching	Mid	205	5.86	6.13	138
Static Batching	High	40	69.0	67.2	1 668

A note on reading the FPS column: “FPS avg” is the mean of the per-frame value $1/\Delta t$, so under variable frame times it weights the cheap frames more heavily and sits above $1000/(\text{mean frame time})$. The high-poly rows are the clearest example, where a 79 ms mean frame time still reports 21 FPS. Wall-clock frame time is therefore the more robust of the two metrics, and the discussion below leans on it whenever the two could disagree.

4.5 Static Batching Degradation with Mesh Complexity

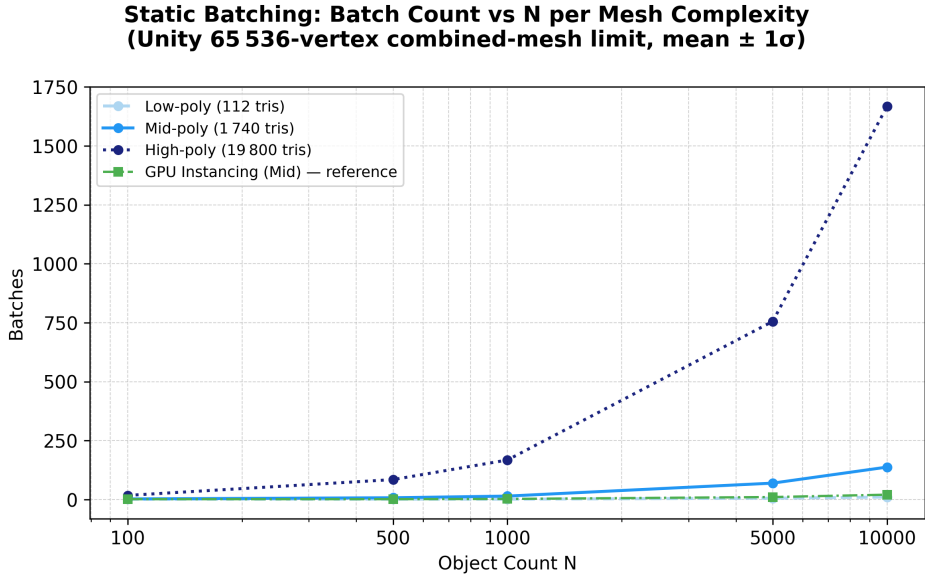


Figure 10: Static Batching batch count (mean $\pm 1\sigma$) vs. N per mesh complexity level, with GPU Instancing (Mid) shown for reference. The 65 536-vertex combined-mesh limit makes the batch count scale as $\lceil N \cdot v / 65\,536 \rceil$.

Figure 10 isolates the most consequential limitation of Static Batching: Unity internally splits the combined mesh at every 65 536-vertex boundary. For high-poly objects of 9 902 vertices each, $N = 10\,000$ produces $\lceil 10\,000 \times 9\,902 / 65\,536 \rceil = 1\,511$ theoretical batches, against a measured 1 668 ($\Delta = +157$, consistent across all 3 repeats). At that point Static Batching offers almost no improvement over Baseline (10 001 batches) and is dramatically worse than GPU Instancing (21 batches).

4.6 Memory Cost

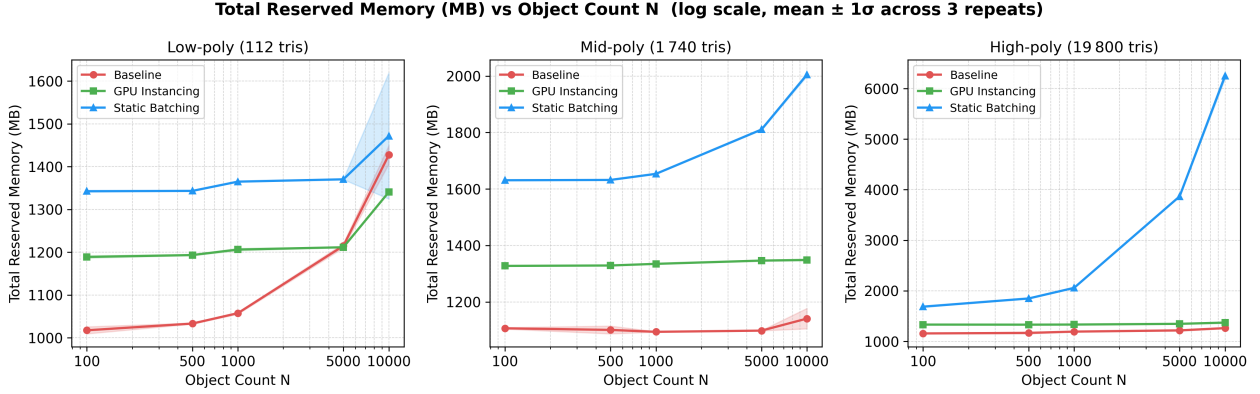


Figure 11: Total reserved memory (MB, mean $\pm 1\sigma$) vs. object count N on a log scale, per mesh complexity. The Static Batching combined mesh makes memory grow linearly with $N \times v$; at $N = 10\,000$ High-poly it reaches about 6 200 MB, against roughly 1 380 MB for GPU Instancing and 1 265 MB for Baseline.

The memory reserved by the Static Batching combined mesh grows in proportion to $N \times v$ (Section 5.3), which imposes a severe cost at high object counts and high mesh complexity. GPU Instancing stores only one mesh plus N transform matrices, a few MB in total, so its memory footprint tracks Baseline across every configuration.

5 Discussion

5.1 Draw-Call Reduction and CPU Relief

The primary hypothesis holds: at large N with low- or mid-poly meshes, GPU Instancing cuts draw calls from $O(N)$ to $O(N/500)$ and measurably lowers the wall-clock frame time. At $N = 10\,000$ low-poly the frame time drops from 8.80 ms under Baseline to 3.64 ms under GPU Instancing, a 59% reduction. Static Batching with low-poly meshes posts the best absolute frame rate (365 FPS at $N = 10\,000$), because its combined-mesh batching is most efficient when the per-object vertex count is small enough to pack many objects into a single 65 536-vertex chunk.

5.2 CPU-Bound to GPU-Bound Transition

At high mesh complexity and large N the bottleneck moves from the CPU, where draw-call overhead dominates, to the GPU, where vertex and fragment processing dominate. At $N = 10\,000$ high-poly the wall-clock frame times are 79.1 ms (Baseline), 65.7 ms (GPU Instancing), and 69.0 ms (Static Batching): the three values fall within roughly 20% of one another, a clear sign that the GPU is close to fully saturated. GPU Instancing’s draw-call advantage still buys a 17% frame-time improvement over Baseline (65.7 vs 79.1 ms), but that margin is far narrower than in CPU-bound configurations, where the speedup exceeds $2\times$. The same transition shows up in both the scatter plot (Figure 8) and the bar chart (Figure 9): as complexity and N grow, the GPU frame time approaches the wall-clock frame time.

5.3 Static Batching: Batch-Count and Memory Trade-off

Static Batching wins on frame rate only while the combined mesh fits into very few 65 536-vertex chunks. At $N = 10\,000$ high-poly it issues 1 668 batches and the optimisation is all but nullified. Table 6 breaks down the theoretical $\lceil N \cdot v / 65\,536 \rceil$ against the measured batch counts: the steady

+1 to +2 excess is fixed camera/lighting overhead, while the +157 excess at high-poly $N = 10\,000$ reflects index-buffer overhead at the vertex-limit boundary.

Table 6: Static Batching theoretical vs. measured batch count. $\Delta = \text{measured} - \text{theoretical}$.

Complexity	N	Vertices/obj	Theoretical	Measured	Δ
Low	100	58	1	2	+1
Low	500	58	1	2	+1
Low	1000	58	1	2	+1
Low	5000	58	5	6	+1
Low	10000	58	9	11	+2
Mid	100	872	2	3	+1
Mid	500	872	7	8	+1
Mid	1000	872	14	15	+1
Mid	5000	872	67	70	+3
Mid	10000	872	134	138	+4
High	100	9902	16	18	+2
High	500	9902	76	85	+9
High	1000	9902	152	168	+16
High	5000	9902	756	756	+0
High	10000	9902	1511	1668	+157

Memory is the second half of the trade-off (Figure 11). At $N = 10\,000$ high-poly, Static Batching reserves about 6 200 MB, a $4.5\times$ overhead against the roughly 1 265–1 380 MB of Baseline and GPU Instancing, because $10\,000 \times 9\,902 \approx 99$ million vertices are duplicated in the CPU-side combined mesh. The practical guideline that follows is straightforward: use Static Batching for low-poly-heavy scenes such as foliage and small props, where the combined mesh stays compact, and treat GPU Instancing as the robust default for high-poly objects or for $N \gtrsim 5\,000$.

5.4 Apple Silicon Unified Memory Note

On Apple Silicon the unified-memory architecture means the GPU-side pressure from the Static Batching combined mesh competes directly with CPU memory. That is a meaningful difference from discrete-GPU systems, where a separate VRAM pool exists, and it is the reason the $4.5\times$ memory cost above matters in practice rather than being absorbed silently into a dedicated VRAM budget.

6 Limitations

A few limitations bound how far these conclusions can be carried:

- **Single platform.** All measurements were taken on Apple Silicon Metal. The D3D11 and Vulkan backends may show different batch-count saturation points and different GPU synchronisation behaviour.
- **Built-in Render Pipeline only.** URP, HDRP, and the SRP Batcher use fundamentally different batching mechanisms and are out of scope here.
- **No overdraw, no shadows, no post-processing.** The top-down opaque grid isolates the draw-call effect cleanly but is not representative of a full game frame, in which fragment cost would be far higher.
- **No animation, physics, or audio.** As explained in Section 2, removing these systems keeps the frame budget dominated by rendering, which is what the experiment needs. A real game

carries additional CPU load that would shift the CPU/GPU balance, and quantifying that shift is left to future work.

- **Fixed camera, no frustum culling exercise.** Every object stays inside the frustum, so the per-object culling savings of GPU Instancing, which can skip culled instances, were not measured against Static Batching’s all-or-nothing combined mesh.
- **Editor-mode measurement.** Cross-validation showed harness and Profiler agreement within 0.5 ms, so any EditorLoop overhead is sub-millisecond on this hardware.

7 Conclusion

This study measured the performance impact of GPU Instancing and Static Batching across 135 measurement windows (45 configurations $\times$ 3 repeats) spanning five object counts, three mesh complexities, and three rendering modes on Apple Silicon Metal.

The key findings are:

1. **GPU Instancing** reliably reduces draw calls to $\lceil N/500 \rceil$ batches regardless of mesh complexity, and delivers significant frame-time savings while the scene is CPU-bound. Once mesh complexity and N are high enough for the GPU to become the binding constraint, the benefit narrows to 17% at $N = 10\,000$ High, against roughly 59% at $N = 10\,000$ Low.
2. **Static Batching** beats GPU Instancing for low-poly objects but degrades sharply with mesh complexity because of Unity’s 65 536-vertex combined-mesh limit. At high-poly with $N = 10\,000$ it gives almost no benefit over the unoptimised Baseline and consumes $4.5\times$ more memory.
3. Under **GPU-bound** conditions, all three modes converge toward comparable wall-clock frame times, which confirms that draw-call optimisation pays off only while the CPU is the bottleneck.

For developers targeting Apple Silicon or other tile-based GPU architectures, GPU Instancing is the recommended default for scenes with many identical objects. Static Batching stays viable for low-poly-heavy scenes, but it has to be weighed against its memory cost and its vertex-count scaling behaviour.

References

- [1] A. Pranckevičius, *Batch, Batch, Batch: What Does It Really Mean?*, Game Developers Conference (GDC), San Francisco, CA, 2011.
- [2] Unity Technologies, *GPU Instancing*, Unity Manual, 2022. [Online] Available: <https://docs.unity3d.com/Manual/GPUInstancing.html>
- [3] Unity Technologies, *Draw Call Batching*, Unity Manual, 2022. [Online] Available: <https://docs.unity3d.com/Manual/DrawCallBatching.html>
- [4] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki, and S. Hillaire, *Real-Time Rendering*, 4th ed. Boca Raton, FL: CRC Press, 2018, ch. 18.
- [5] Unity Technologies Community, *Static Batching vs. GPU Instancing – Memory and Performance Trade-offs*, Unity Discussion Forums, 2019. [Online] Available: <https://discussions.unity.com/t/gpu-instancing-and-static-batching/712655>
- [6] Unity Technologies, *Optimizing Draw Calls: The SRP Batcher*, Unity Blog, 2019. [Online] Available: <https://blog.unity.com/engine-platform/srp-batcher-speed-up-your-rendering>